Code

```
#include <iostream>
#include <string>
using namespace std;
// Node class for Music Playlist
class Node {
public:
 string songTitle;
 string artistName;
 Node* prev;
 Node* next;
 Node(string title, string artist) {
 songTitle = title; artistName = artist;
 prev = nullptr;
 next = nullptr;
 }
};
// Doubly Linked List for Music Playlist
class MusicPlaylist {
private:
 Node* head; // head points to the first song
 Node* current; // current points to the currently selected/playing song
public:
 // Constructor
 MusicPlaylist() {
 head = nullptr;
 current = nullptr;
 }
 // Add a new song at the end of the playlist
 void addSong(string title, string artist) {
 // p points to the newly created node
 Node* p = new Node(title, artist);
 // If playlist is empty
 if (head == nullptr) {
 head = p;
 current = head;
 cout << "Song added successfully.\n";
 return;
 }
 // q is used as the traversal pointer
 Node* q = head;
 // Move q to the last node
 while (q->next != nullptr) {
 q = q->next;
 }
 // Connect the new node
 q->next = p;
 p->prev = q; cout << "Song added successfully.\n";
 }
 // Delete a song using song title
 void deleteSong(string title) {
 if (head == nullptr) {
```

```
cout << "Playlist is empty. Nothing to delete.\n";
return;
}
// q is used to search for the song
Node* q = head;
// Search for the song
while (q != nullptr && q->songTitle != title) {
q = q->next;
}
// Song not found
if (q == nullptr) {
cout << "Song not found in the playlist.\n";
return;
}
// p points to the node that will be deleted
Node* p = q;
// Case 1: deleting the only song
if (p == head && p->next == nullptr) {
head = nullptr;
current = nullptr;
delete p;
cout << "Song deleted successfully.\n";
return;
}
// Case 2: deleting the first song
if (p == head) {
head = p->next;
head->prev = nullptr;
// If current was the deleted song
if (current == p) {
current = head;
}
delete p;
cout << "Song deleted successfully.\n";
return; }
// Case 3: deleting a middle or last song
if (p->prev != nullptr) {
p->prev->next = p->next;
}
if (p->next != nullptr) {
p->next->prev = p->prev;
}
// If current was the deleted song
if (current == p) {
// Move current to next song if possible
if (p->next != nullptr)
current = p->next;
else
current = p->prev;
}
delete p;
cout << "Song deleted successfully.\n";
```

```
}
// Display playlist from first song to last song
void displayPlaylist() {
if (head == nullptr) {
cout << "Playlist is empty.\n";
return;
}
// q is used for forward traversal
Node* q = head;
cout << "\n--- Playlist (Forward) ---\n";
while (q != nullptr) {
cout << "Song: " << q->songTitle
<< " | Artist: " << q->artistName;
if (q == current)
cout << " <-- CURRENT";
cout << endl;
q = q->next;
}
}
// Display playlist from last song to first song
void displayReverse() { if (head == nullptr) {
cout << "Playlist is empty.\n";
return;
}
// q is initially at head
Node* q = head;
// Move q to the last node
while (q->next != nullptr) {
q = q->next;
}
cout << "\n--- Playlist (Reverse) ---\n";
// Traverse using prev pointer
while (q != nullptr) {
cout << "Song: " << q->songTitle
<< " | Artist: " << q->artistName;
if (q == current)
cout << " <-- CURRENT";
cout << endl;
q = q->prev;
}
}
// Play the current song
void playCurrentSong() {
if (current == nullptr) {
cout << "No song is currently selected.\n";
return;
}
cout << "\nNow Playing: "
<< current->songTitle
<< " - "
<< current->artistName
<< endl;
```

```
}
// Move current pointer to the next song
void playNextSong() {
if (current == nullptr) {
cout << "Playlist is empty.\n";
return;
}
// current moves using the next pointer
if (current->next != nullptr) { current = current->next;
cout << "Moved to next song.\n";
playCurrentSong();
}
else {
cout << "Already at the last song. No next song available.\n";
}
}
// Move current pointer to the previous song
void playPreviousSong() {
if (current == nullptr) {
cout << "Playlist is empty.\n";
return;
}
// current moves using the prev pointer
if (current->prev != nullptr) {
current = current->prev;
cout << "Moved to previous song.\n";
playCurrentSong();
}
else {
cout << "Already at the first song. No previous song available.\n";
}
}
// Play the complete playlist once
void playCompletePlaylist() {
if (head == nullptr) {
cout << "Playlist is empty.\n";
return;
}
cout << "\n--- Playing Complete Playlist ---\n";
// q starts from head
Node* q = head;
while (q != nullptr) {
cout << "Playing: "
<< q->songTitle
<< " - "
<< q->artistName
<< endl;
q = q->next; }
// After complete playback, current remains at its selected song.
}
// Repeat the complete playlist specified number of times
void repeatAll() {
```

```
if (head == nullptr) {
cout << "Playlist is empty.\n";
return;
}
int repetitions;
cout << "Enter number of repetitions: ";
cin >> repetitions;
if (repetitions <= 0) {
cout << "Invalid number of repetitions.\n";
return;
}
cout << "\n--- Repeat All ---\n";
for (int i = 1; i <= repetitions; i++) {
cout << "\nRepetition " << i << ":\n";
// Restart playback from head
Node* q = head;
while (q != nullptr) {
cout << "Playing: "
<< q->songTitle
<< " - "
<< q->artistName
<< endl;
q = q->next;
}
}
}
// Show currently selected song
void showCurrentSong() {
if (current == nullptr) {
cout << "No current song selected.\n";
return;
}
cout << "\nCurrent Song:\n";
cout << "Title : " << current->songTitle << endl;
cout << "Artist : " << current->artistName << endl;
} };
// Main function
int main() {
MusicPlaylist playlist;
int choice;
string title;
string artist;
do {
cout << "\n======================================\n";
cout << " MUSIC PLAYLIST MANAGER\n";
cout << "======================================\n";
cout << "1. Add Song\n";
cout << "2. Delete Song\n";
cout << "3. Display Playlist\n";
cout << "4. Display Playlist in Reverse\n";
cout << "5. Play Current Song\n";
cout << "6. Play Next Song\n";
```

```
cout << "7. Play Previous Song\n";
cout << "8. Play Complete Playlist\n";
cout << "9. Repeat All\n";
cout << "10. Show Current Song\n";
cout << "11. Exit\n";
cout << "Enter your choice: ";
cin >> choice;
switch (choice) {
case 1:
cin.ignore();
cout << "Enter song title: ";
getline(cin, title);
cout << "Enter artist name: ";
getline(cin, artist);
playlist.addSong(title, artist);
break;
case 2:
cin.ignore();
cout << "Enter song title to delete: ";
getline(cin, title);
playlist.deleteSong(title);
break; case 3:
playlist.displayPlaylist();
break;
case 4:
playlist.displayReverse();
break;
case 5:
playlist.playCurrentSong();
break;
case 6:
playlist.playNextSong();
break;
case 7:
playlist.playPreviousSong();
break;
case 8:
playlist.playCompletePlaylist();
break;
case 9:
playlist.repeatAll();
break;
case 10:
playlist.showCurrentSong();
break;
case 11:
cout << "Exiting Music Playlist Manager...\n";
break;
default:
cout << "Invalid choice. Please try again.\n";
}
} while (choice != 11);
```

```
 return 0;
}



//====================================
      MUSIC PLAYLIST MANAGER
//====================================
//1. Add Song
//2. Delete Song
//3. Display Playlist
//4. Display Playlist in Reverse
//5. Play Current Song
//6. Play Next Song
//7. Play Previous Song
//8. Play Complete Playlist
//9. Repeat All
//10. Show Current Song
//11. Exit
//Enter your choice: 1

//Enter song title: Kesariya
//Enter artist name: Arijit Singh
//Song added successfully.

//Enter your choice: 1
//Enter song title: Apna Bana Le
//Enter artist name: Arijit Singh
//Song added successfully.

//Enter your choice: 1
//Enter song title: Heeriye
//Enter artist name: Jasleen Royal
//Song added successfully.

//Enter your choice: 3

//--- Playlist (Forward) ---
//Song: Kesariya | Artist: Arijit Singh  <-- CURRENT
//Song: Apna Bana Le | Artist: Arijit Singh
//Song: Heeriye | Artist: Jasleen Royal

//Enter your choice: 4

//--- Playlist (Reverse) ---
//Song: Heeriye | Artist: Jasleen Royal
//Song: Apna Bana Le | Artist: Arijit Singh
//Song: Kesariya | Artist: Arijit Singh  <-- CURRENT

/)Enter your choice: 5
```

//Now Playing: Kesariya - Arijit Singh

//Enter your choice: 6
//Moved to next song.

//Now Playing: Apna Bana Le - Arijit Singh

//Enter your choice: 7
//Moved to previous song.

//Now Playing: Kesariya - Arijit Singh

//Enter your choice: 10

//Current Song:
//Title : Kesariya
//Artist : Arijit Singh

//Enter your choice: 8

//--- Playing Complete Playlist ---
//Playing: Kesariya - Arijit Singh
//Playing: Apna Bana Le - Arijit Singh
//Playing: Heeriye - Jasleen Royal

//Enter your choice: 9
//Enter number of repetitions: 2

//--- Repeat All ---

//Repetition 1:
//Playing: Kesariya - Arijit Singh
//Playing: Apna Bana Le - Arijit Singh
//Playing: Heeriye - Jasleen Royal

//Repetition 2:
//Playing: Kesariya - Arijit Singh
//Playing: Apna Bana Le - Arijit Singh
//Playing: Heeriye - Jasleen Royal

//Enter your choice: 2
//Enter song title to delete: Apna Bana Le
//Song deleted successfully.

//Enter your choice: 3

//--- Playlist (Forward) ---
//Song: Kesariya | Artist: Arijit Singh <-- CURRENT
//Song: Heeriye | Artist: Jasleen Royal

//Enter your choice: 11
//Exiting Music Playlist Manager…